

Document Version: 1.0 (a newer version number means an update on the project)

Assignment Date: 3/27/2025, 5:30pm

Due Date: 4/28/2025, 11:59pm

Group Info: Individual assignment - no groups

Objectives:

- **PART 1 (10 PTS):** Understand Unix pipeline and learn executing system commands in Go
- **PART 2 (15 PTS):** Implement a simple word count application in Go
- **PART 3 (25 PTS):** Implement a sequential MapReduce-based word count application in Go
- **PART 4 (25 PTS):** Implement a MapReduce-based Shared Memory Parallel (SMP) word count application in Go
- **PART 5 (25 PTS):** Implement a MapReduce-based Distributed Memory Parallel (DMP) word count application in Go

1 Project Description

In this project, you will implement various versions of the word count application that we have discussed in class using the Go programming language and AWS. Your word count application will count the number of occurrences of each word in a given file, and print the output in a file in decreasing word frequency order. Each line of the output will include a word and its frequency, separated by a single space. If two or more words happen to have the same frequency value, then you will sort them based on the word, in lexicographic order. Please review the MapReduce paper and the related lecture slides for more information. The content of a sample output file can be as follows:

```
the 1595
and 798
to 731
a 681
of 500
in 456
is 435
MICHAEL 365
his 335
I 324
you 297
he 254
with 249
He 248
The 244
on 203
DON 195
are 164
at 154
for 152
HAGEN 151
DAY: 150
CORLEONE 143
SONNY 141
out 128
my 124
as 122
that 122
```

```
INT 116
it 116
have 110
your 109
EXT 107
1945) 105
from 104
into 103
KAY 102
her 102
be 100
You 98
all 97
by 94
they 94
```

As it can be seen in the partial output above, the words are case-sensitive and the lines are sorted by their frequencies first. If two lines have the same frequency, then they are sorted by their words in lexicographic order (i.e. string order). Please note that uppercase letters should come before lowercase letters, as in Unicode. Also, you do not need to strip any punctuation from the words, anything separated by one or more whitespace characters will be considered as a word.

You will need to implement five different versions of the word count application and execute them on AWS to receive full 100 points for this project. You will be given:

- Two input files: `THEGODFATHER.dat` (197.4 KB), the screenplay of the Godfather movie; and `THEGODFATHER1000.dat` (197.4 MB), which is created by merging one thousand `THEGODFATHER.dat` files back to back to make a larger text file.
- Two correct output files: `THEGODFATHER_CORRECT_WC.dat` and `THEGODFATHER1000_CORRECT_WC.dat`, including the correct output content for testing purposes.
- An incomplete prototype `p3.go` program, including the main function implementation and the prototypes of the functions to be implemented by you. Your solution will extend this prototype program by implementing the empty portions.
- Testing program `p3_test.go` to be used for your evaluation. Do not modify this file. You will submit the result of this test run with the `-v` option (`go test -v p3`) in your report. We will also use this test for grading your project.

1.1 Part 1 (10 PTS): Understand Unix pipeline and learn executing system commands in Go

In this part, you will implement the word count application using pure Unix pipeline inside the provided `WordCount_UNIX()` function. You are free to import necessary Go packages and call external functions. The correct pipeline of commands is provided as a Note in Slide 24, Chapter 9: Distributed Computing and Storage. The output of this pipeline should match 100% with the provided correct output files. You can use the provided test utility of Go to check if the output files match. We will test your code using this utility. In order to run Go's tests, use the following on the bash command line:

```
$ go test -v p3
```

and it should generate PASS for the parts that you have implemented, including the execution time of the test in parenthesis. You will include this output inside your report (more info on the report is provided in Section 2.1). HINT: Check the `TestCleanFiles()` function in the provided `p3_test.go` file to see an example of how to execute a bash command in Go.

1.2 PART 2 (15 PTS): Implement a simple word count application in Go

In this part, you will fill the content of the `WordCount_GO()` function, which will implement a simple sequential non-MapReduce based solution for the word count problem. You can use a simple string tokenizer and a hash-table available in Go to implement this part. You will need to sort the content of the hash table at the end to obtain the desired output. Finally, the result should be written inside the output file received from the user. So, this function will need the name of the input and output files. Please see the main function of the provided `p3.go` to understand what user should pass to run each part.

1.3 PART 3 (25 PTS): Implement a sequential MapReduce-based word count application in Go

This part implements the `WordCount_MR_S()` function, which will need the number of mappers and reducers in addition to the input and the output file names, and it will have four main functionalities to be implemented as described below.

1.3.1 Preparing the Data

Since we do not have a distributed file system like GFS, which would divide the data into smaller chunks for us, we will handle this part ourselves. In order to prepare the data for the mappers, you will first need to divide the provided input file into M equal and disjoint chunks, where M is the number of mappers received by the user. When you merge these chunks, you should be able to get the exact original file back. Each mapper will be responsible for a specific chunk.

1.3.2 Mapping the Data

Once the data is divided into M equal chunks, you will go through these chunks *sequentially* to generate the key-value pairs ($\langle \text{word}, 1 \rangle$ for word count) to be written inside the intermediate files to be generated by you. These intermediate files will be used by the reducers later. For each chunk, your mapper function will generate R intermediate files, where R is the number of reducers received by the user. At the end, there will be $M \times R$ intermediate files generated sequentially, where each reducer will be responsible for M specific intermediate files. The aim in generating R separate intermediate files for each chunk is being able to map a specific key occurring in each chunk into a specific reducer. For instance, the word "the" may occur in all the chunks, and all the key-value pairs including the key "the" should be reduced by a single reducer. In order to determine which key-value pair goes to which intermediate file, you can pass the key (word) to a hash function and based on the hash value generated, you can determine the reducer and its intermediate file ID by using a simple mod operation.

As an example, assume we have 4 mappers and 2 reducers. Since we have 4 mappers, the input file will be divided into 4 equal chunks by the data preparation step: *chunk0*, *chunk1*, *chunk2*, *chunk3*. Then, your mappers will generate 8 intermediate files, 2 for each chunk. For *chunk0*, files `int0-0` and `int0-1` will be generated, for *chunk1*, `int1-0` and `int1-1` will be generated, and so on. All intermediate files ending with 0 (`int0-0`, `int1-0`, `int2-0`, `int3-0`) will be reduced by *reducer0*

and all the intermediate files ending with 1 (`int0-1`, `int1-1`, `int2-1`, `int3-1`) will be reduced by *reducer1*. So, if the word "the" appears in all chunks, and the operation `hash("the")%R` returns 0, then the intermediate files including the key-value pairs with the key "the" will be `int0-0`, `int1-0`, `int2-0`, and `int3-0`, and these are the intermediate files to be reduced by *reducer0*. This way, you can consistently map the same key to the same reducer.

1.3.3 Reducing the Data

The task of the reducer is going through M specific intermediate files that are assigned to itself, and aggregating the values of the keys that are encountered in these intermediate files. Once all the values are aggregated for each key, the results will be written inside a single intermediate *results* file for each reducer. At the end, there will be R intermediate *results* files to be combined by the combiner. As in the mapper case, reducers will execute *sequentially* for this part.

1.3.4 Combining the Data

Since we will have R separate intermediate *results* files, we need another module to combine and sort them. The combiner will go through R intermediate *results* files and combine them into a single final *results* file. The combiner is also responsible for sorting the final results before writing them inside the final *results* file as specified by the sorting instructions.

1.4 PART 4 (25 PTS): Implement a MapReduce-based Shared Memory Parallel (SMP) word count application in Go

This part is the same as the previous part, except the mappers and the reducers will run in parallel in a shared memory machine using multi-threading. You will use goroutines to achieve multi-threading in Go. The data will be prepared sequentially as in the previous part, but once the data is ready, your mapper threads will execute in parallel using goroutines. Next, you will need to join all mapper threads to make sure that they are all completed and it is safe to start the reducer threads. Once the mapper threads are joined, you will execute the reducer threads in parallel and you will also need to join them as in the mapper case so that the combiner can start safely. Once the reducers are joined, you will finally combine the intermediate files sequentially as in the previous part. Your implementation for this part will go inside the `WordCount_MR_SMP()` function.

1.5 PART 5 (25 PTS): Implement a MapReduce-based Distributed Memory Parallel (DMP) word count application in Go

Your implementation will go inside the `WordCount_MR_DMP()` function. First, you need a shared file system for input, intermediate, and output file sharing among your EC2 instances. For this purpose, you will use the Amazon Elastic File System¹. The first three steps of the following tutorial will provide you the necessary directions to create a shared file system:

- <https://docs.aws.amazon.com/efs/latest/ug/getting-started.html>

This shared storage space should be available to all your instances. **All your tests will be done using Ubuntu Server 24.04 t2.xlarge instances with 4 vCPUs and 16 GB memory.** At the bare minimum case, you will need to create two instances to observe execution time benefits, and your

¹<https://aws.amazon.com/efs/>

solution will be divided into two multi-threaded MapReduce solutions that you have implemented in the previous part to be executed in parallel on two different EC2 Ubuntu Server 24.04 t2.xlarge instances. The aim for this part is testing the scalability of the DMP solution since SMP solution will not scale well once the number of mappers/reducers reach the number of cores. You will need to analyze the execution time performance of this solution in your report for scalability purposes.

You will need to implement a master that will be responsible for preparing the data, combining the data, and starting the mapper and the reducer tasks. Master will communicate with the mappers and the reducers through Remote Procedure Calls (RPC) as in the example provided in the class website. Once the data is ready in shared EFS folder, the master can trigger the mappers that will be executing on different machines using multiple threads as in the SMP solution. Once the mappers are completed, they need to inform the master. This will simulate joining threads in a distributed memory system. Next, the master can trigger the execution of the reducers, and similarly reducers can inform the master about their completion. Once all the reducers are completed, the master can combine/sort the data and generate the final output file.

In order to achieve execution time benefit for the DMP solution over the SMP solution, you should have a least two instances. One instance can run the master and half of the mappers/reducers in SMP mode, and the other instance can run the other half of the mappers/reducers in SMP mode. For example, if the number of mappers and the reducers are both set to 8, then machine 1 will run the master and 4 of the mappers/reducers in a multi-threading mode, and machine 2 will run the remaining 4 mappers/reducers in a multi-threading mode. So, mappers and reducers will run in different machines in parallel. To observe a better speed-up, you can use more machines.

Collaboration and Code Sharing Policy

This course assumes that assignments submitted by students will be generated by the students themselves working individually. This policy indicates the following constitute violations of academic honesty: a student has another person/entity do the work of any portion of a graded assignment for them, which includes purchasing work from a company, hiring a person or company to complete an assignment or exam, **and/or fully/partially copying code from other resources (such as Internet) or generating code using generative AI tools (such as ChatGPT)**. Except parts provided by us or the book, if any, all submitted code must be written by *you* and must not be shared. You are not allowed to go over your classmates' or friends' code, and they are not allowed to see your code. You are only allowed to have high-level verbal discussions about the project with other students. In addition, you are not allowed to publish your code partially (in forums or other sites for asking questions) or fully (in public version control services such as GitHub, etc.). **You cannot post your code on your public GitHub account, unless you make it private!** You are also not allowed to share your code with future students of this course. **You may NOT use Chegg.com, CourseHero.com, MyAssignmentHelp.com, or any other sites to post your assignment/code or receive any help.** Not complying with these collaboration and code sharing rules will put you and those with whom you share your code under the risk of plagiarism for this or the following semesters. Cheating/copying of assignments (including Internet resources) will be reported to the dean's office for plagiarism and a grade of F will be recorded for the course. **Just check with the instructor in advance if you are not sure if a certain practice would put you under the risk of plagiarism. A simple MS Teams chat with the instructor will suffice!**

2 Submission

Submission will be done through Blackboard strictly following the instructions below. Your work will be penalized 5 points out of 100 if the submission instructions are not followed.

2.1 What to Submit

1. `p3.go` including the source code of your project.
2. `report.pdf` including the following information (write the question first, then your answer underneath):
 - **Question 1:** Your name and ID.
 - **Question 2:** Did you fully implement the project as described? If not, which parts are not implemented at all, or which parts are not implemented by following the specified description?
 - **Question 3:** Provide screenshots proving the execution on AWS, which includes your test run output (`go test -v p3`). **Make sure that your go tests are run on Ubuntu Server 24.04 t2.xlarge instance with 4 vCPUs and 16 GB memory.** Briefly discuss the scalability of different parts implemented by you. Consider the following issues while discussing the scalability:
 - (a) How are the execution times of the different parts compared to each other for the small input size?
 - (b) What happens when the input file size gets larger?
 - (c) What is your envision for the execution time of different parts when the input size gets even larger, into gigabyte, terabyte, or even petabyte range?
 - (d) What is the scalability limitation of the SMP solution, and how DMP solution can eliminate this limitation?
 - (e) Which solution would you go with if the input file size was small?
 - (f) Which solution would you go with if the input file size was large?
 - (g) Which solution would you go with if the input file size was small, but there were many input files to be processed?

How to Submit

1. Create a directory and name it as the combination of the project number and your UofL student ID number. For example, if a student's ID is 1234567, then the name of the directory will be `ProjectX-1234567` for project X.
2. Put all the files to be submitted (only the ones asked in the *What to Submit* section above) into this directory.
3. Zip the directory. As a result, you will have the file `ProjectX-1234567.zip`.
4. Upload the file `ProjectX-1234567.zip` to Blackboard using the "Attach File" option. You do not need to write anything on the "Submission" and "Comments" sections.
5. You are allowed to make multiple submission attempts but only the last attempt submitted before the deadline will be graded. **NO LATE SUBMISSIONS WILL BE GRADED!**

3 Grading

Grading of your program will be done by an automated process by running the provided Go test. Your program will also be passed through a copy-checker program which will examine the source codes if they are original or copied; we have a database of similar solutions from previous years as well as from the Internet. We will also examine your source file(s) manually.

4 Updates

- No updates.